

# CompanyGraph — What stays when the model changes

Code, System One, System Two and a person: which model you keep, and which you can replace.

RESTS ON concept Instance

Robert Blust · Software Engineer & Architect

01

# “Model” names five things

Two of them are models that learned. Three of them are files you keep.

RESTS ON

concept Core

concept Instance

concept Type

SYSTEM TWO

**Opus, Fable, Gemini Flash** writes and reasons

SYSTEM ONE

**TypeSafe’s Jev** answers typed questions

META-MODEL

**CompanyGraph core** the vocabulary and its rules

MENTAL MODEL

**one company’s instance** its facts, in that vocabulary

GRAPH

**what is drawn from it** a site, a server, a chat

# Four layers, one rule

Each decision goes to the lowest layer that can make it.

RESTS ON

decision Agents write the model, and a person approves every change to it

4 **Person**  
approves

3 **System Two**  
writes and reads · Opus, Fable, Gemini Flash

2 **System One**  
answers typed questions · Jev

1 **Code**  
decides what code can decide · the check

# Fixed: the meta-model

The vocabulary, and the rules for writing each page, in Markdown, at a release an instance pins.

RESTS ON `concept Schema`

`decision The schema written as prose is the only schema`

`decision A release is its tag, and nothing we make is published to a registry`

meta/core/feature-schema.md

## ## Writing rules

- The tagline says what someone can now do, in their words, and never names a screen, a service or a vendor.
- The H1 names the capability, not the vendor that supplies it.
- The page says nothing about a release, a ticket or a delivery date: those move, and a feature outlives all three.

# Fixed: the mental model

A company's facts as Markdown in git, checked against the release it adopted. No tool and no model owns it.

RESTS ON `concept Instance`

`strategic-objective` A company's model belongs to no tool, and any graph database can hold it

`decision` Every entity carries an id that outlives its name

model/features/...writing-rules.md

---

`id:` 01a10770-fd86-7891-a8a0-5ea8e270b7f2

`products:` CompanyGraph Core

`concepts:` Agent pass, Schema, Rule, Instance

---

**# A decision model asks each page its schema's writing rules**

> A company learns which of its pages most likely break a writing rule before anyone reads them...

4 Person

3 System Two

2 System One

1 Code

# Code: the check

References, schemas, ids. The same answer every run, and no model asked.

RESTS ON `concept Check` `feature Checks an instance runs`

```
decision A reference resolves by its declared type, never by name alone
```



```
$ companygraph check
✓ model/ against meta/core at core <release>: the mechanical checks pass
  not checked: every `## Writing rules` in every schema – that is the
  agent pass, R0
✓ Markdown files in the one form
```

# System One: the judge

4 Person

3 System Two

2 System One

1 Code

Each writing rule becomes a typed question. Jev answers with a choice and a probability. The owner says yes to exactly what leaves the machine.

It advises, and changes nothing.

RESTS ON

feature A decision model asks each page its schema's writing rules

question Does our data leave our hands?

```
$ companygraph judge --consent <digest>
judge: advisory – it gates nothing and is no pass

model/features/checkout.md

! 0.31 r1 The tagline says what someone can now do, in their words,
and never names a screen, a service or a vendor.
```

# System Two: writes and reads

4 Person

3 System Two

2 System One

1 Code

Skills draft. Validate reads the judge's flags first, then every page. The server and the chat answer, and name what each answer rests on.

RESTS ON `concept Agent pass`

`feature` An agent does the work no script can, by the skills an instance ships

`decision` The server only reads, and answers at one named commit

companygraph-validate

**Mechanical checks** passed

**Writing rules**

`model/features/checkout.md` breaks feature `r1`: "The tagline says what someone can now do ..."

**Not checked:** nothing

4 Person

3 System Two

2 System One

1 Code

# A person approves

A pass that finds nothing approves nothing. The Owner's word merges.

RESTS ON `rule The Owner's word merges`

`strategic-objective` Every gate an agent's work passes is held by a person

`control` A reviewer reads the output, not the report

pull request

companygraph / companygraph ✓

conventions / conventions ✓

merge on the Owner's word

# The flow

Each step does what it is good at, and says what it leaves to the next.



RESTS ON

concept Check

feature A decision model asks each page its schema's writing rules

concept Agent pass

# Fixed and replaceable

What stays is what you wrote. What changes is what reads it, and each says whether it is replaceable today or by design.

RESTS ON

decision Claude is the first agent an instance supports, and not the only one

strategic-objective A company keeps its model with whichever agent it chooses

question Does it only work with Claude, or can we use ChatGPT or Gemini?

## FIXED

the meta-model

your mental model

the check

your rules and decisions

a person's approval

## REPLACEABLE

the agent that reads, over MCP

today

the editor

today

the System One model

by design

the System Two model that writes

by design

the graph database

by design

# Measure before you swap

Every model is pinned. A new one is measured against the same company before it is named.

RESTS ON `kpi Grounded Answer Rate`

question `Can I trust what this chat says?`

rule `A check counts only when its output was seen`

bin/judges/typesafe.mjs

```
// The one place the tooling names a judge ... so a second judge is a second
// file beside this one and the questions do not change. The model is pinned
// rather than `jev-latest`, because the band the report flags by is measured
// against one model, and a newer one is measured again before it is named here.
```

# Six practices

- 01 Keep knowledge where no model owns it.
- 02 Let code decide what code can decide.
- 03 Ask typed questions before free text.

RESTS ON `value Run on what we publish`

- 04 A model proposes, a person decides.
- 05 Pin every model, and measure before you move it.
- 06 Ask before data leaves the machine.

# What stays

The model you will change most often is the one you should depend on least.

Then the next model is a pin you move, not a project you start.

RESTS ON

decision Agents write the model, and a person approves every change to it

4 **Person**  
approves

3 **System Two**  
writes and reads · Opus, Fable, Gemini Flash

2 **System One**  
answers typed questions · Jev

1 **Code**  
decides what code can decide · the check

# Credits

INSPIRED BY

TypeSafe, who call their decision model Jev a System One model, beside the System Two models that generate text.

SYSTEM ONE AND SYSTEM TWO

Daniel Kahneman, *Thinking, Fast and Slow*, 2011; first named by Keith Stanovich and Richard West, 2000

JEV AND THE API THE JUDGE CALLS

[typesafe.ai](https://typesafe.ai) · [docs.typesafe.ai](https://docs.typesafe.ai)

THE MODELS IN THIS TALK

[companygraph/meta-model](https://companygraph/meta-model) · [companygraph/mental-model](https://companygraph/mental-model)